

Reviewer Pack — Minimal Hodge-Tate Degree and SAT Clause Subset Complexity C...

Entry ada0f867de9b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 20:55:22 UTC

Minimal Hodge-Tate Degree and SAT Clause Subset Complexity Correlation

Entry ID: ada0f867de9b **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 20:55:22 UTC

1. Conjecture

Field A (mathematical branch): Arithmetic Geometry (Hodge-Tate Theory) **Field B** (complexity object): Boolean Satisfiability (SAT Clause Subset Complexity)

Statement:

For every CNF φ with n variables, the minimal Hodge-Tate degree of its associated algebraic variety is linearly correlated with its SAT clause subset complexity, such that $\text{htd}(\varphi) = \Theta(\text{csc}(\varphi))$ for some function csc .

Rationale (proposer's reasoning):

Hodge-Tate theory provides a way to study algebraic varieties over finite fields, and it has been used in arithmetic geometry. By associating an algebraic variety with a CNF, we can potentially uncover new insights into the complexity of SAT clause subsets. This conjecture suggests that the geometric properties of these varieties could be related to the difficulty of finding satisfying assignments for φ .

Taxonomy category: ArithmeticGeometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ee411780c13fb76d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between minimal Hodge-Tate degree and SAT clause subset complexity exceeds 0.7 for at least 80% of all seeds, with no seed showing a correlation below -0.5.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hodge-Tate degree" AND "SAT clause subset complexity" - "arithmetic geometry" AND "Boolean satisfiability problem" - "minimal Hodge-Tate degree" related TO "SAT clause subset complexity"

Top relevant hits considered: - [<http://arxiv.org/abs/cs/0608100v1>] Similarity of Semantic Relations - [<http://arxiv.org/abs/0810.1207v1>] A Layered Grammar Model: Using Tree-Adjoining Grammars to Build a Common Syntactic Kernel for Related Dialects - [<http://arxiv.org/abs/1310.8154v3>] Characteristic cohomology of the infinitesimal period relation

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=4.6s

5.1 Generated Python source

```
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def extended_gcd(a, b):
    if a == 0:
        return (b, 0, 1)
    else:
        g, x, y = extended_gcd(b % a, a)
        return (g, y - (b // a) * x, x)

def mod_inverse(a, m):
    g, x, _ = extended_gcd(a, m)
    if g != 1:
        raise ValueError("Modular inverse does not exist")
    else:
        return x % m

def matrix_mod_inv(matrix, mod):
    n = len(matrix)
    adj = [[0 for _ in range(n)] for _ in range(n)]
    det = 0
```

```

for i in range(n):
    for j in range(n):
        minor = [row[:j] + row[j+1:] for row in matrix[:i] + matrix[i+1:]]
        adj[j][i] = ((-1) ** (i+j)) * determinant(minor)
det = determinant(matrix)
if det == 0:
    raise ValueError("Matrix is singular")
inv_det = mod_inverse(det, mod)
for i in range(n):
    for j in range(n):
        adj[i][j] = (adj[i][j] * inv_det) % mod
return adj

def determinant(matrix):
    n = len(matrix)
    if n == 1:
        return matrix[0][0]
    elif n == 2:
        return matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0]
    else:
        det = 0
        for j in range(n):
            minor = [row[:j] + row[j+1:] for row in matrix[1:]]
            det += ((-1) ** j) * matrix[0][j] * determinant(minor)
        return det

def gaussian_elimination(matrix, mod):
    n = len(matrix)
    augmented_matrix = [row[:] + [i] for i, row in enumerate(matrix)]
    for i in range(n):
        pivot_row = max(range(i, n), key=lambda x: abs(augmented_matrix[x][i]))
        augmented_matrix[i], augmented_matrix[pivot_row] = augmented_matrix[pivot_row], augmented_matrix[i]
        lead = augmented_matrix[i][i]
        for j in range(i, n + 1):
            augmented_matrix[i][j] = (augmented_matrix[i][j] * mod_inverse(lead, mod)) % mod
        for k in range(n):
            if k != i:
                factor = augmented_matrix[k][i]
                for j in range(i, n + 1):
                    augmented_matrix[k][j] = (augmented_matrix[k][j] - factor * augmented_matrix[i][j]) % mod
    return [row[:-1] for row in augmented_matrix]

def generate_cnf(n):
    clauses = []
    for i in range(2**n):
        clause = []
        for j in range(n):
            if (i >> j) & 1:
                clause.append(j + 1)
            else:
                clause.append(-(j + 1))
        clauses.append(clause)
    return clauses

def sat_clause_subset_complexity(cnf):
    n = len(cnf[0])

```

```

max_clauses = 2**n
complexity = [0] * (max_clauses + 1)
for i in range(1, max_clauses + 1):
    for j in range(i):
        if all(x in cnf[j] for x in cnf[i]):
            complexity[i] += 1
return sum(complexity)

def hodge_tate_degree(cnf):
    n = len(cnf[0])
    matrix = [[0 for _ in range(n)] for _ in range(n)]
    for clause in cnf:
        for x in clause:
            if x > 0:
                i, j = x - 1, (x - 1) % n
            else:
                i, j = -x - 1, (-x - 1) % n
            matrix[i][j] += 1
    matrix = gaussian_elimination(matrix, 2)
    rank = sum(1 for row in matrix if any(row))
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    cnf = generate_cnf(n)
    htd = hodge_tate_degree(cnf)
    csc = sat_clause_subset_complexity(cnf)
    correlation = (htd - csc) / math.sqrt(htd**2 + csc**2)
    return {
        "metric_name": "Pearson Correlation Coefficient",
        "metric_value": correlation,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": abs(correlation) > 0.7 and correlation >= -0.5,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(30)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
    elif any(r["conjecture_holds"] and r["metric_value"] < -0.5 for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if result["conjecture_holds"] == False)
        print(f"RESULT: FALSIFIED counterexample='correlation_below_minus_0.5' first_failing_seed={first_failing_seed}")

```

```

else:
    print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ac681755.py", line 143, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ac681755.py", line 126, in run_trial
    htd = hodge_tate_degree(cnf)
          ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ac681755.py", line 118, in hodge_tate_degree
    matrix = gaussian_elimination(matrix, 2)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ac681755.py", line 78, in gaussian_elimination
    augmented_matrix[i][j] = (augmented_matrix[i][j] * mod_inverse(lead, mod)) % mod
                              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ac681755.py", line 36, in mod_inverse
    raise ValueError("Modular inverse does not exist")
ValueError: Modular inverse does not exist

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete the computation to determine the Pearson correlation coefficient. | next: Investigate and fix the error in the test code that caused it to crash. Once fixed, rerun the test to verify if the conjecture meets the pre-registered support condition.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14573
2	propose	ollama_remote	glm4:latest	0	0	13323
3	preregistration	ollama_remote	glm4:latest	0	0	9182
4	novelty	ollama_remote	glm4:latest	0	0	11198
5	novelty	ollama_remote	glm4:latest	0	0	9636

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	28130
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11147
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7867
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17193
10	judge	ollama_remote	glm4:latest	0	0	19935

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 142184 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/ada0f867de9b.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/ada0f867de9b.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/ada0f867de9b.tar.gz* (if generated)